VEB DASTURLAR XAVFSIZLIGI

A3. Mijoz tomoni dasturlash xavfsizligi: JavaScript, DOM va Browser API

1-VARIANT — DOMga xavfsiz chiqarish, Fetch API javoblari va brauzer saqlash vositalarini tekshirish

Mashg‘ulot maqsadi. Talaba foydalanuvchi matnining DOMga qanday o‘tishini kuzatadi, xavfli chiqarish nuqtasini topadi va tuzatadi. Fetch orqali olingan javobning statusi, turi va tuzilmasini tekshiradi; localStorage hamda sessionStorage xususiyatlarini tajribada solishtiradi; sahifaga ulangan skriptning imkoniyatlarini baholaydi.

Kutiladigan natija. Talaba DOM XSSni sahifadagi nazorat yozuvi orqali isbotlaydi, textContent yordamida bartaraf etadi va qayta sinaydi. HTTP 400, noto‘g‘ri JSON tuzilmasi va tarmoq uzilishida interfeysning qanday javob berishini ko‘rsatadi. Brauzerda saqlangan token nima sababdan JavaScriptdan himoyalanmaganini tushuntiradi.

Ko‘rsatkich	Qiymat
Semestr / variant	Faqat 5-semestr / A3 / 1-variant
Tavsiya etilgan vaqt	80 daqiqa; tayyor VM va oldindan ko‘chirilgan 4 ta kod fayli bilan
Kompyuter xonasi	30 ta Windows 11; 8 yadroli CPU, 32 GB RAM, 1 TB SSD; lokal tarmoq va internet
Virtual muhit	VMware Workstation; Ubuntu Desktop 24.04 LTS; 4 vCPU, 8 GB RAM, obrazdagi disk
Laboratoriya manzili	http://127.0.0.1:8003/ — server va brauzer bitta Ubuntu VM ichida
Ish shakli	Har bir talaba o‘z kompyuterida; individual hisobot va qisqa himoya
Baholash	10 ball; asosiy tajribalar va dalillar asosida

0 Talaba bajaradigan dastlabki tayyorgarlik

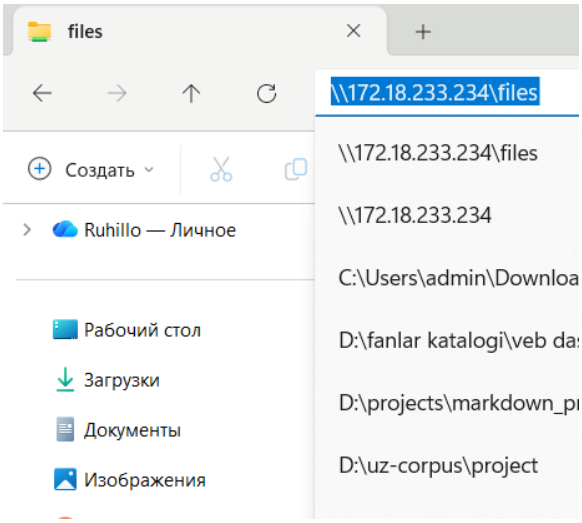
O‘qituvchi ishlaydigan tayyor VMware obrazini fayl serverga joylashtiradi. VMware o‘rnatish, obrazni nusxalash, hisoblar, proksi, paketlar, kod, sinov, snapshot va hisobotni talaba mustaqil bajaradi. Dastlabki tayyorgarlik 60–90 daqiqa yoki zarur qo‘shimcha vaqt oladi; asosiy 80 daqiqaga kirmaydi.

0.1 Fayl serverga Windows orqali kirish

Windowsda Win+R yoki File Explorer manzil satrida quyidagi SMB yo‘lni oching. Bu brauzer uchun HTTP manzil emas.

```
\\172.18.233.234\files
```

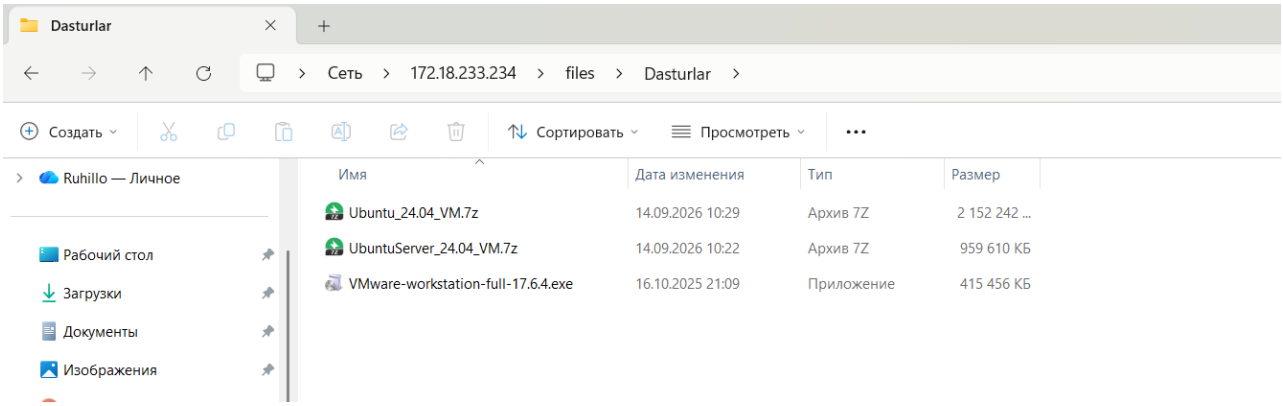
Windows Security oynasida login fileuser va parol 1234567 kiriting. Bu ma’lumotlar faqat SMB ulanishi uchun; Ubuntu yoki proksi paroli emas. Ubuntu ichida obraz bilan berilgan sudo huquqli hisobdan foydalaning.



1-rasm. Fayl serverga kirish oynasi

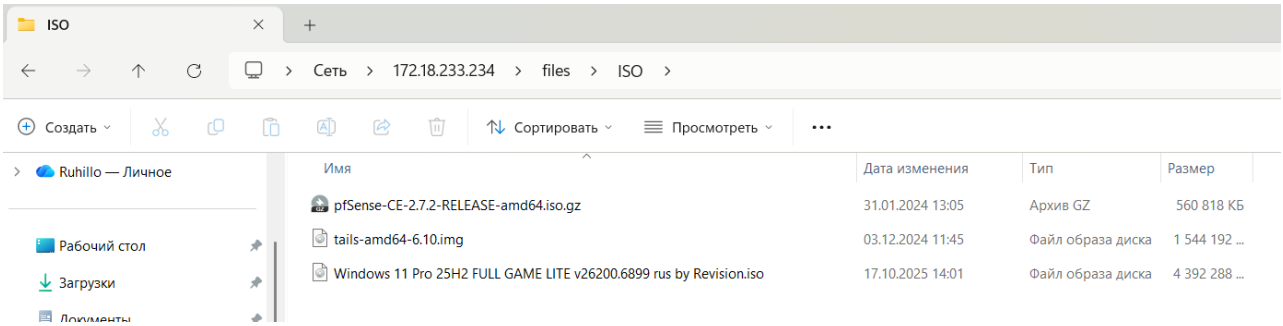
0.2 Dasturlar va tayyor obrazni tanlash

Dasturlar papkasidan VMware-workstation-full-17.6.4.exe va Ubuntu_24.04_VM.7z fayllarini Windowsning lokal diskiga nusxalang. Ubuntu_24.04_VM.7z — brauzerli Desktop; ushbu fan ishlari uchun shu obraz tanlanadi. UbuntuServer_24.04_VM.7z fayli Server muhitiga tegishli.



2-rasm. Dasturlar papkasidagi VMware va Ubuntu obrazlari

ISO papkasi serverda mavjud, lekin bu ishda Linuxni ISO orqali o‘rnatish bajarilmaydi. Boshqa operatsion tizim talab qilinsa, keyin joylashtiriladigan tegishli tayyor VMware obrazi olinadi; uning nomi oldindan taxmin qilinmaydi.



3-rasm. ISO papkasining ko‘rinishi

0.3 VMware va individual nusxani ochish

1. VMware o‘rnatgichini Windows administrator huquqi bilan ishga tushiring. O‘rnatish tugagach zarur bo‘lsa kompyuterni qayta yuklang. Obraz arxivini 7-Zip orqali D:\Veb-A3-LAB\VM papkasiga to‘liq oching; arxiv ichidan yoki tarmoq papkasidan VMni ishga tushirmang.

2. VMware → File → Open orqali ochilgan papkadagi .vmx faylini tanlang. VM nomini Veb-A3-LAB deb belgilang. Birinchi ishga tushishda “I copied it” tanlansin; har talabaning nusxasi va MAC manzili alohida bo‘ladi.

3. VM o‘chiq holatda Settings orqali 4 vCPU va 8 GB RAM belgilang. Obrazdagi virtual disk hajmini saqlang, kichraytirmang. Diskda ochilgan obraz va snapshot uchun yetarli joy qoldiring. Brauzer, curl va server bir Ubuntu VM ichida ishlaydi.

4. Ubuntu konsoliga kirib terminalni oching. Windowsdagi 127.0.0.1 VMga olib bormaydi. Quyidagi buyruqlarni Ubuntu Bashda bajaring:

```
sudo hostnamectl set-hostname veb-a3-lab
whoami
id
sudo -v
date -Is
```

0.4 Vaqtinchalik NAT va universitet proksisi

VM Settings → Network Adapter → NAT tanlang; paketlar yuklanayotganda Connected yoqilgan bo‘lsin. Bridged, port forwarding va Windows Internet Connection Sharing yoqilmaydi. 172.18.1.1:2500 — proksi manzili, VM IPsi yoki gatewayi emas.

Har yangi paket o‘rnatish terminalida quyidagi export buyruqlarini qayta bajaring. APT uchun alohida doimiy konfiguratsiya keyingi blokda berilgan.

```
export http_proxy="http://172.18.1.1:2500"
export https_proxy="http://172.18.1.1:2500"
export HTTP_PROXY="http://172.18.1.1:2500"
export HTTPS_PROXY="http://172.18.1.1:2500"
export no_proxy="localhost,127.0.0.1,::1,app.a6.test,evil.a6.test,a9.test,wrong.test"
export NO_PROXY="$no_proxy"
curl -I https://ubuntu.com
```

APT sozlamasi uchun sudo nano /etc/apt/apt.conf.d/99proxy buyrug‘ini bajaring. Mavjud fayl bo‘lsa avval mazmunini tekshiring; quyidagi uch satrni saqlang:

```
Acquire::http::Proxy  "http://172.18.1.1:2500";
Acquire::https::Proxy "http://172.18.1.1:2500";
Acquire::ftp::Proxy   "http://172.18.1.1:2500";
```

99proxy faylini saqlang va nano oynasidan chiqing. Quyidagi buyruqlar fayl mazmuni emas; ularni Ubuntu terminalida bajaring:

```
apt-config dump | grep -i proxy
sudo apt update
sudo apt install -y python3 curl nano open-vm-tools open-vm-tools-desktop
```

0.5 Firefox va mashg‘ulot paketlarini tayyorlash

Firefoxni menyudan ochib tekshiring. Mavjud bo‘lsa qayta o‘rnatmang. Firefox yo‘q bo‘lsa, APT va terminal exportlari snapd xizmatiga avtomatik o‘tmasligini hisobga olib, quyidagi xizmat konfiguratsiyasini yarating:

```
sudo apt install -y snapd
sudo mkdir -p /etc/systemd/system/snapd.service.d
sudo nano /etc/systemd/system/snapd.service.d/veb-proxy.conf
```

veb-proxy.conf fayliga faqat quyidagi uchta Environment satri va [Service] sarlavhasini yozing:

```
[Service]
Environment="HTTP_PROXY=http://172.18.1.1:2500"
Environment="HTTPS_PROXY=http://172.18.1.1:2500"
Environment="NO_PROXY=localhost,127.0.0.1,::1"
```

Faylni saqlab, muharrirdan chiqing. Endi terminalda xizmatni yangilang va Firefoxni o‘rnating:

```
sudo systemctl daemon-reload
sudo systemctl restart snapd
sudo snap install firefox
firefox --version
```

Keyingi muhitni tayyorlash bo‘limidagi apt va pip buyruqlarini NAT hali ulangan paytda bajaring. pip ishlatilgan variantda .venvni faollashtiring; sudo pip ishlatmang. requirements-lock.txt va haqiqiy versiyalarni saqlang. Yuklash xato bersa proksi/NAT va vaqtni tekshiring; paketlar tayyor bo‘lmaguncha tajribani boshlamang.

0.6 Lokal tajribaga o‘tish va dalillarni chiqarish

Barcha paketlar va kodlar tayyorlangach VMni o‘chiring. Har bir Network Adapter uchun Connected va Connect at power on belgilarini olib tashlang. Alohida laboratoriya adapteri kerak emas: barcha xizmatlar 127.0.0.1da ishlaydi. VMni qayta boshlang; Firefox → Settings → Network Settings → No proxy tanlang. curl sinovlarida --noproxy "*" ishlatiladi. A6dagi lokal domen nomlari saqlanadi.

VM o‘chiq paytda VM → Snapshot → Take Snapshot orqali Veb-A3-base yarating. Kod va konfiguratsiyani hujjatdagi boshlang‘ich rejimga tayyorlang. Snapshotga qaytishdan oldin hisobot va dalillarni Windowsga eksport qiling. Ish oxirida tiklashni talaba o‘zi bajaradi; snapshot nusxa olish o‘rnini bosmaydi.

Windowsda D:\Veb-A3-LAB\Report papkasini yarating. VM Settings → Options → Shared Folders → Always enabled → Add orqali faqat shu papkani VebA3 nomi bilan ulang. Ubuntu ichida quyidagilarni bajaring:

```
vmware-hgfsclient
sudo mkdir -p /mnt/veba3share
sudo vmhgfs-fuse .host:/VebA3 /mnt/veba3share \
  -o allow_other,uid=$(id -u),gid=$(id -g)
touch /mnt/veba3share/export-check.txt
```

vmware-hgfsclient chiqishida ulashilgan nom, Windows papkasida export-check.txt ko‘rinsin. Mountpoint oldin ulangan bo‘lsa qayta mount qilmang. Hisobot, kerakli kod va dalillarni shu katalogga nusxalang; token, cookie va yopiq kalitlarni umumiy papkaga chiqarmang.

P1 — tayyor obraz va VMware nusxasi; P2 — proksi/APT va paket versiyalari; P3 — uzilgan adapterlar hamda lokal xizmat; P4 — snapshot va Shared Folders dalillarini hisobotga qo‘shing. Kod bloklari och kulrang, muhim manzil va sozlamalar och sariq fon bilan ajratilgan.

Asosiy amaliy mashg‘ulot

Zarur dasturlar va oldingi bilimlar

1. Python 3 — mahalliy HTTP server; standart kutubxona yetarli, pip paketlari talab qilinmaydi.
2. Firefox va uning Developer Tools vositalari — DOM, Console, Network va Storage tekshiruvi; curl va nano — so‘rov yuborish va kod tahriri.
3. A1dagi HTTP va origin tushunchalari, A2dagi kiritma tekshiruvi hamda xavfsiz chiqarish asoslari; oddiy HTML va JavaScript sintaksisi.

A1–A2 bilan davomiylik. Oldingi Ubuntu VMdan foydalaning. A3 alohida ~/websec/A3 papkasida va 8003-portda ishlaydi. Har ish joyi o‘z loopback manziliga ega; 30 kompyuterda bir xil portdan foydalanish mumkin.

Asosiy tushunchalar va tajriba chegarasi

JavaScript, DOM va ishonch chegarasi

JavaScript kodini brauzer bajaradi; DOM esa sahifaning xotiradagi tuzilmasidir. Hodisa tinglovchisi tugma bosilganda ishlaydi. Foydalanuvchi kiritmasi, API javobi va brauzer omboridagi qiymat dasturga kelgan ma’lumotdir: ularni kod sifatida talqin qilish xavfli. [1, 2]

Bu ishda manba (source) — textarea.value; xavfli chiqarish nuqtasi (sink) — innerHTML. innerHTML matnni HTML sifatida tahlil qiladi. textContent oddiy matn tugunini hosil qiladi; matn ichidagi teg va hodisa atributlarini bajarmaydi. Bu himoya oddiy matn chiqarishga mos, URL yoki JavaScript kontekstining barcha muammolariga umumiy yechim emas. [1]

Amal	Mazmuni	Ushbu ishdagi qoida
element.innerHTML = value	HTML sifatida tahlil qilish	Faqat ataylab zaif tajriba nuqtasida
element.textContent = value	Oddiy matn chiqarish	DOM va API natijalarini chiqarishda
fetch() + await	Asinxron HTTP so‘rovi	Status, Content-Type va tuzilmani tekshirish
localStorage / sessionStorage	Origin doirasidagi mijoz omborlari	Maxfiy bo‘lmagan sinov qiymatlari bilan
script src	Sahifa kontekstida skript bajarish	Manba ishonchliligi va yaxlitligini baholash

Fetch va brauzer omborlari

Fetch — AJAX usulida sahifani to‘liq qayta yuklamasdan HTTP so‘rovi yuborish vositasi. Promise va await natija kelishini kutadi, brauzer esa boshqa hodisalarni qayta ishlashi mumkin. 400 yoki 404 javobi fetch Promise’ini avtomatik rad etmaydi; response.ok alohida tekshiriladi. finally tugmani qayta faollashtiradi. [3]

localStorage origin bo‘yicha ajratiladi va odatda brauzer qayta ochilganda saqlanadi. sessionStorage origin hamda tab sessiyasi bo‘yicha ajratiladi; sahifa yangilansa saqlanadi. Ikkalasiga ham shu origin kontekstida ishlayotgan JavaScript kira oladi. sessionStorage tokenni XSSdan yashirmaydi. [4, 5, 6]

Laboratoriya chegarasi

4. Tajriba faqat 127.0.0.1:8003dagi o‘quv sahifasida bajariladi; serverni LAN yoki internetga ochmang.
5. Zaif DOM nuqtasi va badshape javobi ataylab kiritilgan. Sinov faqat DOM-XSS yozuvini hosil qiladi; haqiqiy akkaunt yoki token ishlatilmaydi.
6. CSP, autentifikatsiya va server avtorizatsiyasini joriy etish keyingi mashg‘ulotlarda ko‘riladi. Bu ishdagi lokal HTTP server ishlab chiqarish muhiti uchun mo‘ljallanmagan. [9]

1.1. Muhitni tayyorlash va fayllarni joylashtirish

0-bo‘limdagi tayyor Ubuntu_24.04_VM.7z obrazidan foydalaning. VMda shu hujjatdagi vCPU va RAM qiymatlarini belgilang; obrazning mavjud virtual diskini saqlang. Ubuntu ichida Ctrl+Alt+T orqali Bash terminalini oching.

0-bo‘limdagi NAT va proksi ulangan tayyorgarlikda quyidagi paketlarni o‘rnating. Firefox 0-bo‘limda tekshiriladi. Kodlar tayyorlangach NATni uzing.

```
sudo apt update
sudo apt install -y python3 curl nano
mkdir -p ~/websec/A3
cd ~/websec/A3
python3 --version
firefox --version
```

Hujjat oxiridagi 1-ilovadan server.py, 2-ilovadan index.html va vendor.js, 3-ilovadan app.js fayllarini ko‘chiring. Har bir kod bloki o‘z nomidagi alohida UTF-8 faylga yoziladi. Sahifa sarlavhasi va izohlarni fayl ichiga kiritmang. Python chekinishlarini, JavaScript qo‘shtirnoqlarini va HTML teglarini saqlang.

```
nano server.py
nano index.html
nano app.js
nano vendor.js
python3 -m py_compile server.py
ls -l server.py index.html app.js vendor.js
python3 server.py
```

nano muharririda Ctrl+O, Enter bilan saqlang, Ctrl+X bilan chiqing. Server terminalida A3: http://127.0.0.1:8003/ yozuvi chiqadi. Uni ochiq qoldiring. Ikkinchi terminalni Ctrl+Alt+T orqali ochib, ish papkasiga o‘ting:

```
cd ~/websec/A3
curl --noproxy "*" -i http://127.0.0.1:8003/
curl --noproxy "*" -i "http://127.0.0.1:8003/api/echo?q=Salom"
```

Kutiladigan natija: ikkala so‘rov 200; API javobi application/json va {"text": "Salom"}. Ubuntu ichidagi Firefoxda http://127.0.0.1:8003/ manzilini oching. Sahifada sinov matni, DOM tugmalari, Fetch bo‘limi va vendor skripti holati ko‘rinadi.

1.2. Tekshirish vositalarini ochish

7. F12 orqali Developer Toolsni oching. Inspector — DOM, Console — JavaScript xabarlari, Network — HTTP, Storage — Web Storage uchun ishlatiladi.
8. Network oynasida Disable Cache imkoniyatini yoqing. Sahifani Ctrl+Shift+R bilan yangilang; app.js va vendor.js statusi 200 bo‘lsin.
9. index.htmlni ikki marta bosib file:// orqali ochmang. Windowsdagi 127.0.0.1 Ubuntu VMning loopback manzilidan boshqa muhitga tegishli.

Talaba 0-bo‘lim bo‘yicha tayyor Ubuntu Desktop VMware obrazini ochadi, universitet proksisi orqali paketlarni o‘rnatadi va ilovalardagi kod fayllarini o‘zi yaratadi. Nusxa olish, paket yuklash va kod tayyorlash asosiy 80 daqiqadan oldin bajariladi; dastlabki tayyorgarlikka 60–90 daqiqa yoki zarur qo‘shimcha vaqt ajrating.

1.3. DOMga zaif va xavfsiz chiqarishni taqqoslash

Sinov matniga Salom, A3! yozing. “Zaif chiqarish” va “Xavfsiz chiqarish” tugmalarini ketma-ket bosing. Oddiy matnda natija o‘xshash. Endi A3 kiriting: zaif tugmada A3 qalin ko‘rinadi, xavfsiz tugmada teglar bilan birga oddiy matn chiqadi.

Inspector orqali preview elementini ko‘ring: birinchi holatda b elementi, ikkinchi holatda matn tuguni mavjud. Tegning qalin ko‘rinishi HTML talqin qilinganini bildiradi; JavaScript bajarilganini isbotlash uchun keyingi tajriba kerak.

1.4. DOM XSSni nazorat belgisi bilan aniqlash

Quyidagi bir qatorli sinov matnini textarea ichiga to‘liq kiriting. Uni Console oynasida bajarmang. “Zaif chiqarish” tugmasini bosing:

```
<img src="/a3-missing.png" onerror="document.getElementById('flag').textContent='DOM-XSS'">
```

Kutiladigan natija: marker TAYYOR o‘rniga DOM-XSS bo‘ladi. /a3-missing.png mavjud emas, rasm so‘rovi 404 qaytaradi, onerror hodisasi esa DOMdagi yozuvni o‘zgartiradi. Networkdagi ushbu 404 tajribaning kutilgan qismidir.

Bu yerda oqim aniq: textarea.value → preview.innerHTML → img elementi → onerror → flag.textContent. Manba ishonchsiz, innerHTML uni HTMLga aylantirgan. Sinov qiymati serverga saqlanmagan; DOM o‘zgarishi mijozdagi kod yo‘li orqali yuz bergan.

Endi ayni matn bilan “Xavfsiz chiqarish” tugmasini bosing. resetDOM eski elementni olib tashlaydi, marker TAYYORga qaytadi; to‘liq img matni ko‘rinadi. Inspector ichida preview ostida img elementi bo‘lmasin. Networkni tozalab xavfsiz tugmani yana bosing: yangi a3-missing.png so‘rovi chiqmasligi kerak.

Dalilni to‘g‘ri talqin qilish

10. D1: bir xil sinov matnida zaif holatdagi DOM-XSS va xavfsiz holatdagi TAYYOR markerlarini, tegning matn sifatida chiqarilishini saqlang.
11. DOM XSS dalili — tugma orqali kelgan matndagi hodisa kodining bajarilishi. Console orqali qo‘lda marker o‘zgartirish bunday dalil hisoblanmaydi.
12. innerHTML orqali kiritilgan script tegi odatda avtomatik bajarilmaydi; shu sababli bu tajribada hodisa atributli img ishlatilgan. Bitta samarasiz script sinovi sahifa xavfsizligini isbotlamaydi. [1]

Mustaqil tekshiruv

Oddiy matnga A & B < C yozib ikkala chiqarishni solishtiring. Xavfsiz holatda belgilar yo‘qolmasligini qayd eting. Himoya foydalanuvchi ma’lumotini butunlay o‘chirib yuborish orqali emas, uni tegishli matn kontekstida chiqarish orqali ishlayotganini tushuntiring.

1.5. Fetch API: javob va xatoliklarni tekshirish

“DOMni tozalash”ni bosing. Sinov matniga A3 kiriting, Networkni tozalang va “API so‘rovi”ni bosing. /api/echo so‘rovini ochib Query String Parameters, javob statusi, Content-Type hamda Response qismini tekshiring.

Kutiladigan natija: HTTP 200; JSON text maydoni A3; sahifada teglar oddiy matn sifatida ko‘rinadi. URLSearchParams so‘rov parametrini URL formatiga kodlaydi, lekin bu HTMLdan himoya emas. API natijasi textContent bilan chiqarilgani uchun HTMLga aylanmaydi. [3]

Dastlabki DOM XSS sinov matnini “API so‘rovi” orqali ham yuboring. Marker TAYYOR bo‘lib qoladi, API natijasida img matni ko‘rinadi; api elementi ostida yangi img hosil bo‘lmaydi.

Sinov kiritmasi / holat	HTTP / javob	Interfeysda kutiladigan natija
Salom, A3!	200; text — satr	Muvaffaqiyat: HTTP 200; matn chiqadi
Bo‘sh textarea	400; error maydoni	Xato: HTTP 400; oldingi natija tozalanadi
badshape	200; {"text": 123}	Xato: Javob tuzilmasi noto‘g‘ri.
Server vaqtincha o‘chirilgan	HTTP javobi olinmaydi	Xato: …; tarmoq xabari brauzerga bog‘liq

Bajarish tartibi

13. Matnni butunlay o‘chirib API tugmasini bosing. 400 javobining o‘zi fetch uchun tarmoq xatosi emas; !response.ok sharti uni xato yo‘liga o‘tkazadi.

14. badshape yozib yuboring. Server laboratoriya uchun ataylab noto‘g‘ri tur qaytaradi: status 200 bo‘lsa ham text maydoni satr emas. typeof tekshiruvi buni ushlaydi.
15. Server terminalida Ctrl+C bosing. Sahifani yangilamasdan API tugmasini bosing. Xato ko‘rinib, tugma qayta faollashishini tekshiring. So‘ng python3 server.py bilan serverni qayta ishga tushiring va Salom yuboring.

Koddagi AbortController uch soniyalik kutish chegarasini beradi. Tarmoq uzilishi darhol aniqlansa vaqt tugashi xabari chiqmasligi mumkin. Sinovda aniq inglizcha xato matniga emas, xato holati ko‘rsatilishi va tugma tiklanishiga baho bering.

1.6. Boshqariladigan nosozlik: javob turini almashtirish

server.py nusxasini saqlang. API uchun return self.reply satridagi application/json; charset=utf-8 qiymatini vaqtincha text/plain; charset=utf-8 ga o‘zgartiring; serverni qayta ishga tushiring. Salom so‘rovida 200 bo‘lsa ham “JSON javob kutilgan edi.” xatosi chiqsin. Keyin asl Content-Typeni tiklang va qayta sinang.

```
cp server.py server.original.py
nano server.py
# Har o‘zgarishdan keyin serverni Ctrl+C bilan to‘xtatib qayta ishga tushiring.
```

D2: 200 muvaffaqiyat, 400, noto‘g‘ri tuzilma va noto‘g‘ri Content-Type natijalarini jadvalda solishtiring. MIME turi, JSON sintaksisi va maydon turi alohida tekshiruv ekanini tushuntiring.

1.7. localStorage va sessionStorage bilan tajriba

Sahifada F12 → Console oynasini oching. Quyidagi qisqa, tushunarli buyruqlarni kiriting; bu boshqariladigan saqlash tajribasi. Faqat sun’iy qiymatlar ishlatiladi:

```
localStorage.setItem('a3_theme', 'dark');
sessionStorage.setItem('a3_tab', 'birinchi');
localStorage.setItem('a3_demo_token', 'DEMO-A3-NOT-SECRET');
localStorage.getItem('a3_theme');
sessionStorage.getItem('a3_tab');
```

Storage bo‘limida http://127.0.0.1:8003 originini tanlang. Local Storage ichida a3_theme va a3_demo_token, Session Storage ichida a3_tab ko‘rinsin. Sahifani yangilang: dark va birinchi qiymatlari saqlanadi.

Ctrl+T bilan butunlay yangi tab ochib, manzilni qo‘lda kiriting. Eski tabni “Duplicate” qilmang va window.open ishlatmang: opener mavjud bo‘lsa sessionStorage boshlang‘ich nusxasi ko‘chishi mumkin. Yangi tab Console oynasida yuqoridagi ikki getItemni bajaring. localStorage → dark; sessionStorage → null bo‘lishi kutiladi. [5, 6]

Tekshiruv	Birinchi tab	Yangi, mustaqil tab
localStorage.getItem('a3_theme')	dark	dark
sessionStorage.getItem('a3_tab')	birinchi	null
localStorage.getItem('a3_demo_token')	DEMO-A3-NOT-SECRET	DEMO-A3-NOT-SECRET

1.8. Ulangan skriptning imkoniyatini aniqlash

Token yozilgandan keyin sahifani yangilang. “Uchinchi tomon skripti maketi” bo‘limida vendor skripti o‘qigan qiymat DEMO-A3-NOT-SECRET bo‘ladi. vendor.js faylini oching: unda localStorage.getItem chaqiruvi bor. Bu skript tashqi kutubxona maketi sifatida lokal fayldan yuklangan; hech qayerga ma’lumot yubormaydi.

Oddiy script src orqali yuklangan kod hujjatning JavaScript kontekstida bajariladi; uni boshqa faylga ajratish izolyatsiya yaratmaydi. CDNdan olingan klassik skript ham sahifaga kiritilgach shu imkoniyatlarga ega bo‘lishi mumkin. Shu sababli kutubxona manbasini tanlash, versiyasini nazorat qilish va keraksiz skriptlarni olib tashlash muhim. [7]

```
localStorage.removeItem('a3_demo_token');
location.reload();
```

Yangilangandan keyin vendor bo‘limida YOQ chiqadi. Bu tokenni saqlamaslik natijasi; vendor skriptining barcha imkoniyatlari cheklangan degani emas. Xuddi shunday token sessionStoragega yozilsa, shu tabda bajarilgan JavaScript uni ham o‘qiy oladi.

Himoya xulosasi. Brauzer omborida tokenni Base64ga o‘tkazish yoki qiymat nomini yashirish uni XSSdan himoyalamaydi. HttpOnly cookie JavaScript orqali cookie qiymatini o‘qishni cheklaydi, lekin faol XSSning foydalanuvchi nomidan so‘rov yuborish imkoniyatini to‘liq yo‘qotmaydi. Sessiya va CSRF himoyasi keyingi ishlarda ko‘riladi. [4]

D3: omborlarning ikki tabdagi farqi va vendor bo‘limining token bor/yo‘q holatini saqlang. Hisobotda originning sxema, host va portdan tashkil topishini qayd eting.

1.9. Zaif nuqtani tuzatish va qayta sinash

app.js nusxasini saqlang. “Zaif chiqarish” tugmasining hodisa tinglovchisidagi quyidagi satrni almashtiring:

```
cp app.js app.original.js
nano app.js
```

Avvalgi satr:

```
preview.innerHTML = input.value; // Deliberately unsafe lab sink.
```

Tuzatilgan satr:

```
preview.textContent = input.value;
```

Faylni saqlang va brauzerda Ctrl+Shift+R bosing. Statik app.js keyingi so‘rovda qayta o‘qiladi; faqat shu fayl o‘zgarganda Python serverini qayta ishga tushirish shart emas. Tugma nomi taqqoslash uchun saqlanadi, lekin endi uning chiqarish amali tuzatilgan.

16. DOM XSS sinov matnini aynan o‘sha “Zaif chiqarish” tugmasi bilan qayta tekshiring. Marker TAYYOR qolishi, img elementi yaratilmasligi kerak.
17. Salom, A3! hamda A & B < C qiymatlari to‘g‘ri ko‘rinsin. Fetch muvaffaqiyat va badshape tekshiruvi saqlangan bo‘lsin.
18. D4: avvalgi va tuzatilgan satr, bir xil sinov matni va qayta tekshirish dalillarini qo‘shing. Faqat tugmani yashirish yoki nomini o‘zgartirish tuzatish hisoblanmaydi.

1.10. SRI orqali fayl yaxlitligi — ixtiyoriy qo‘shimcha

Asosiy ish tugagach vendor.jsning SHA-384 qiymatini hisoblang. Bu bosqich 80 daqiqaga kiritilmagan; taxminan 10–15 daqiqa kerak. [8]

```
cd ~/websec/A3
cp vendor.js vendor.original.js
python3 - <<'PYHASH'
from pathlib import Path
import hashlib, base64
digest = hashlib.sha384(Path('vendor.js').read_bytes()).digest()
print('sha384-' + base64.b64encode(digest).decode())
PYHASH
```

Chiqqan sha384-… qiymatini index.htmldagi vendor script tegiga integrity atributi sifatida to‘liq qo‘ying. Quyidagi HISOBLANGAN qiymati o‘rniga o‘z natijangiz yoziladi:

```
<script src="vendor.js" integrity="HISOBLANGAN" defer></script>
```

Ctrl+Shift+R bilan yangilang: to‘g‘ri xeshda vendor ishlaydi. Keyin vendor.js oxiriga faqat // A3 change izohini qo‘shing; HTMLdagi xeshni o‘zgartirmang. Yana yangilang: Console yaxlitlik xatosini ko‘rsatadi, vendor bo‘limida “Skript hali ishlamadi.” qoladi. Server 200 qaytarishi skript bajarilganini bildirmaydi.

```
cp vendor.original.js vendor.js
```

Asl faylni tiklab yangilang; skript yana ishlasin. SRI kutilgan baytlar bilan moslikni tekshiradi, skriptning foydali yoki zararli ekanini hal qilmaydi. Bu lokal, bir originli namuna; boshqa origindan yuklashda CORS talablari ham bajarilishi kerak. [8]

2. Variant topshiriqlari va natijalarni qayd etish

Har bir natijani o‘z tajribangiz bilan to‘ldiring. Kutiladigan natijani o‘lchov sifatida ko‘chirmang. “Mos emas” holatida sabab va qayta tekshirish natijasini yozing.

№	Sinov	Kutiladigan natija	Amaldagi natija / dalil
1	Boshlang‘ich sahifa va API	200; JS fayllari yuklangan	_____
2	A3: zaif / xavfsiz	b elementi / oddiy matn	_____
3	DOM XSS: zaif tugma	DOM-XSS; img elementi	_____
4	DOM XSS: xavfsiz tugma	TAYYOR; img elementi yo‘q	_____
5	Fetch: DOM sinov matni	200; matn chiqadi; kod bajarilmaydi	_____
6	Fetch: bo‘sh kiritma	400; xato; natija maydoni bo‘sh	_____
7	Fetch: badshape	200; tuzilma xatosi	_____
8	API MIME turini buzish	200; JSON turi xatosi	_____
9	Serverni to‘xtatish / tiklash	Xato; tugma tiklanadi / 200	_____
10	Omborlar: yangi mustaqil tab	local: dark; session: null	_____
11	Vendor: token bor / o‘chirilgan	DEMO-A3-NOT-SECRET / YOQ	_____
12	Zaif sink tuzatilgandan keyin	TAYYOR; matn to‘liq chiqadi	_____
13	SRI: o‘zgargan / tiklangan fayl	Bloklanadi / ishlaydi; ixtiyoriy	_____

Tahliliy topshiriqlar

19. Manba → chiqarish nuqtasi → brauzer amali zanjirini o‘z kodingizdagi nomlar bilan yozing. Qaysi satr ishonch chegarasini buzganini belgilang.
20. Nega URLSearchParams va JSON ishlatilishining o‘zi DOM XSSni to‘xtatmasligini 2–3 gap bilan tushuntiring.
21. localStorage o‘rniga sessionStorage ishlatish qaysi saqlanish farqini beradi va qaysi XSS xavfini saqlab qolishini yozing.
22. Ulangan kutubxona uchun uchta chora taklif qiling: masalan, ishonchli manba, versiyani belgilash, SRI yoki keraksiz bog‘liqlikni olib tashlash. Har birining cheklovini ayting.

Ish yakunida

a3_theme va a3_tabni tegishli omborlardan removeItem bilan o‘chiring; a3_demo_token qolmaganini tekshiring. Serverni Ctrl+C bilan to‘xtating. Tuzatilgan 4 fayl va hisobotni saqlang; snapshotga qaytishdan oldin nusxani VMdan tashqariga ko‘chiring.

3. Hisobot, vaqt taqsimoti va baholash

Hisobot fayli: A3_1_Familiya_Guruh.docx. Unda mavzu, maqsad, VM va brauzer versiyasi, bajarilgan qadamlar, to‘ldirilgan natija jadvali, kod tuzatishi va qisqa xulosa bo‘lsin. Ekran tasvirlariga D1–D4 belgisi hamda tushuntiruvchi izoh bering.

Dalil	Talab qilinadigan mazmun
D1 — DOM	Bir xil matn: zaif bajarilish va xavfsiz matn chiqarish; marker va Inspector
D2 — Fetch	Muvaffaqiyat, 400, badshape va noto‘g‘ri Content-Type; Network va interfeys
D3 — Storage / vendor	Mustaqil tablar farqi; vendorning sun’iy qiymatni o‘qishi va qiymat o‘chirilishi
D4 — tuzatish	innerHTML → textContent; avvalgi sinovning tuzatilgan tugmada qaytarilishi

80 daqiqalik tavsiya etilgan reja

Bosqich	Daqiqa	Yakuniy natija
Muhitni tekshirish va ishga tushirish	8	Sahifa, JS va API ishlaydi
DOM va nazoratli XSS sinovi	18	D1 dalillari
Fetch va javoblarni buzib tekshirish	18	D2; server turi tiklangan
Storage va vendor tajribasi	14	D3 dalillari
Kod tuzatishi va qayta sinov	12	D4; regressiya tekshiruvi
Hisobot va qisqa himoya	10	Jadval, xulosa, fayllar topshirilgan

Baholash mezonlari — jami 10 ball

Mezon	Ball
Muhit va boshlang‘ich HTTP tekshiruvi	1,0
DOM manbasi, sink va XSS dalilini to‘g‘ri izohlash	2,0
Fetch: status, MIME, tuzilma va tarmoq xatosini tekshirish	2,0
Storage farqi va vendor imkoniyatlarini dalil bilan ko‘rsatish	1,5
Zaif kodni tuzatish va aynan shu yo‘lni qayta sinash	2,0
Hisobotning aniqligi, xulosa va og‘zaki himoya	1,5

SRI qo‘shimcha mashqi asosiy 10 ballni olish uchun shart emas. Qisman bajarilgan mezonda to‘g‘ri dalil va tushuntirishga mutanosib ball qo‘yiladi. Dalilsiz “ishladi” xulosasi to‘liq bajarilish hisoblanmaydi.

Himoya savollari

23. innerHTML va textContent orasidagi asosiy farq nima?
24. Nega 400 javobida fetch catch tarmog‘iga o‘zi o‘tmaydi?
25. 200 javob qachon ishlovga yaroqsiz bo‘lishi mumkin?
26. sessionStorage tokenni XSSdan yashiradimi?
27. SRI to‘g‘ri bo‘lgan vendor skripti localStorageni o‘qiy oladimi?

Talaba: _____ Guruh: _____ Sana: _____
O‘qituvchi qaydi: _____
Ball: _____ / 10

4. Nosozliklarni bartaraf etish va manbalar

Tezkor tekshirish

28. Sahifa ochilmaydi: brauzer Ubuntu VM ichida ekanini va 127.0.0.1:8003 manzilini tekshiring. Server terminalidagi xabarni o‘qing; ss -ltnp bilan portni tekshiring.
29. Tugmalar ishlamaydi: Console ichidagi birinchi SyntaxErrorni tuzating. app.js va vendor.js nomlari, ASCII qo‘shtirnoqlar hamda qavslar to‘g‘ri ko‘chirilgan bo‘lsin; JS so‘rovlari 200 qaytarsin.
30. Marker o‘zgarmaydi: avval zaif kod tiklanganini, to‘liq img sinov matni textarea ichiga kiritilganini tekshiring. U Console uchun buyruq emas; xavfsiz tugmada marker o‘zgarmasligi to‘g‘ri natija.
31. Kod yangilanmaydi: Ctrl+Shift+R bosing. server.py o‘zgarsa serverni qayta ishga tushiring; app.js o‘zgarsa brauzer qayta yuklashi yetarli.
32. Storage natijasi boshqacha: localhost bilan 127.0.0.1ni aralashtirmang; port 8003 bo‘lsin. Yangi tabni Ctrl+T bilan oching; shaxsiy rejim va saqlashni cheklovchi sozlamalarni qayd eting.
33. Vendor YOQ ko‘rsatadi: sun’iy tokenni aynan a3_demo_token kaliti bilan yozib, sahifani yangilang. SRI sinovidan keyin fayl o‘zgargan bo‘lsa asl vendor.jsni tiklang.

Foydalanilgan manbalar

Mavzu foydalanuvchi taqdim etgan “Veb dasturlar xavfsizligi fani mavzulari.docx”dagi A3 va M3ga asoslandi. Tuzilish avvalgi A1–A2 hamda “Kompyuter_tarmoqlari_A6_1_variant_batafsil.docx” namunasi bilan uyg‘unlashtirildi. Rasmiy texnik manbalarga murojaat: 07.09.2026.

[1] OWASP. DOM based XSS Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

[2] OWASP. Cross Site Scripting Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[3] MDN. Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

[4] OWASP. HTML5 Security Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[5] MDN. Window: localStorage property. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[6] MDN. Window: sessionStorage property. https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage

[7] OWASP. Third Party Javascript Management.
https://cheatsheetseries.owasp.org/cheatsheets/Third_Party_Javascript_Management_Cheat_Sheet.html

[8] MDN. Subresource Integrity. https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Subresource_Integrity

[9] Python. http.server. https://docs.python.org/3/library/http.server.html

1-ilova. server.py — mahalliy HTTP va JSON server

Quyidagi kodni server.py fayliga to‘liq yozing. Faqat belgilangan uchta statik fayl va /api/echo yo‘li xizmat qiladi; papka ro‘yxati tarqatilmaydi.

```
import json
from pathlib import Path
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlsplit, parse_qs

ROOT = Path(__file__).resolve().parent
FILES = {'/': ('index.html', 'text/html; charset=utf-8'),
         '/app.js': ('app.js', 'text/javascript; charset=utf-8'),
         '/vendor.js': ('vendor.js', 'text/javascript; charset=utf-8')}

class Handler(BaseHTTPRequestHandler):
    def reply(self, status, body, kind):
        self.send_response(status)
        self.send_header('Content-Type', kind)
```

```
        self.send_header('Content-Length', str(len(body)))
        self.send_header('Cache-Control', 'no-store')
        self.send_header('X-Content-Type-Options', 'nosniff')
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        url = urlsplit(self.path)
        if url.path in FILES:
            name, kind = FILES[url.path]
            return self.reply(200, (ROOT / name).read_bytes(), kind)
        if url.path == '/api/echo':
            q = parse_qs(url.query).get('q', [''])[0]
            status, data = 200, {'text': q}
            if not 1 <= len(q) <= 200:
                status, data = 400, {'error': '1-200 belgi kiriting.'}
            elif q == 'badshape':
                data = {'text': 123}
            body = json.dumps(data, ensure_ascii=False).encode('utf-8')
            return self.reply(status, body, 'application/json; charset=utf-8')
        self.reply(404, b'Not found', 'text/plain; charset=utf-8')

if __name__ == '__main__':
    print('A3: http://127.0.0.1:8003/', flush=True)
    HTTPServer(('127.0.0.1', 8003), Handler).serve_forever()
```

badshape faqat javob tuzilmasini tekshirish tajribasi uchun qo‘shilgan. CSP bu o‘quv serverida o‘rnatilmagan, shuning uchun zaif DOM hodisasini kuzatish mumkin. Uni haqiqiy xizmat uchun tayyor konfiguratsiya deb qabul qilmang.

2-ilova. index.html va vendor.js

A. index.html — quyidagi blok bitta HTML faylga yoziladi.

```
<!doctype html>
<html lang="uz">
<meta charset="utf-8">
<title>A3: mijoz tomoni xavfsizligi</title>
<style>
body {font: 18px sans-serif; max-width: 850px; margin: 30px auto;}
textarea {width: 95%; height: 75px;}
section {border: 1px solid #aaa; padding: 15px; margin: 15px 0;}
pre {white-space: pre-wrap; overflow-wrap: anywhere;}
button {margin: 4px; padding: 8px;}
</style>
<h1>A3: DOM, Fetch va Web Storage</h1>
<p>Faqat mahalliy o'quv tajribasi. Haqiqiy token kiritmang.</p>
<label for="input">Sinov matni:</label>
<textarea id="input">Salom, A3!</textarea>
<section>
<h2>1. DOM</h2>
<button id="weak">Zaif chiqarish</button>
<button id="safe">Xavfsiz chiqarish</button>
<button id="reset">DOMni tozalash</button>
<p>Marker: <strong id="flag">TAYYOR</strong></p>
<div id="preview"></div>
</section>
<section>
<h2>2. Fetch API</h2>
<button id="fetch">API so'rovi</button>
<p id="status">So'rov yuborilmadi.</p>
<pre id="api"></pre>
</section>
<section>
<h2>3. Uchinchi tomon skripti maketi</h2>
<p id="vendor">Skript hali ishlamadi.</p>
```

```
</section>
<script src="app.js" defer></script>
<script src="vendor.js" defer></script>
</html>
```

B. vendor.js — ushbu blok alohida JavaScript fayliga yoziladi.

```
'use strict';
// Local mock: no network request and no real credential.
const demo = localStorage.getItem('a3_demo_token');
document.getElementById('vendor').textContent =
  "Vendor skripti o‘qigan qiymat: " + (demo || 'YOQ');
```

vendor.js tashqi kutubxona maketi. U faqat sun’iy qiymatni shu sahifada ko‘rsatadi, tarmoq orqali uzatmaydi. defer skriptlarni hujjat tahlili tugagach, teglar tartibida bajarishga imkon beradi.

3-ilova. app.js — DOM va Fetch tajribalari

Quyidagi kod dastlabki, ataylab zaif DOM satrini ham o‘z ichiga oladi. 1.9-bosqichda aynan shu satr tuzatiladi.

```
'use strict';
const byId = id => document.getElementById(id);
const input = byId('input');
const preview = byId('preview');
function resetDOM() {
  preview.replaceChildren();
  byId('flag').textContent = 'TAYYOR';
}
byId('weak').addEventListener('click', () => {
  resetDOM();
  preview.innerHTML = input.value; // Deliberately unsafe lab sink.
});
byId('safe').addEventListener('click', () => {
  resetDOM();
  preview.textContent = input.value;
});
byId('reset').addEventListener('click', resetDOM);

async function loadAPI() {
  const button = byId('fetch');
  button.disabled = true;
  byId('api').textContent = '';
  byId('status').textContent = 'Yuklanmoqda...';
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 3000);
  try {
    const params = new URLSearchParams({q: input.value});
    const response = await fetch('/api/echo?' + params, {
      signal: controller.signal, credentials: 'omit', cache: 'no-store'
    });
    if (!response.ok) throw new Error('HTTP ' + response.status);
    const kind = (response.headers.get('Content-Type') || '')
      .split(';')[0].trim().toLowerCase();
    if (kind !== 'application/json') {
      throw new Error('JSON javob kutilgan edi.');
    }
    const data = await response.json();
    if (!data || typeof data.text !== 'string') {
      throw new Error("Javob tuzilmasi noto‘g‘ri.");
    }
    byId('api').textContent = data.text;
    byId('status').textContent = 'Muvaffaqiyat: HTTP 200';
  } catch (error) {
    const message = error.name === 'AbortError'
      ? 'Kutish vaqti tugadi.' : error.message;
    byId('status').textContent = 'Xato: ' + message;
```

```
  } finally {
    clearTimeout(timer);
    button.disabled = false;
  }
}
byId('fetch').addEventListener('click', loadAPI);
```